T2

September 6, 2025

1 Genetic Algorithm (GA)

Objetivo Implementar un Algoritmo Genético (AG) que incorpore elitismo y compare el desempeño de distintos operadores de selección: Ruleta, Muestreo Estocástico Universal (SUS) y Torneo. El propósito es analizar y contrastar sus efectos sobre convergencia, calidad de solución y estabilidad.

Requisitos de implementación

1) Elitismo (obligatorio) Integre un esquema de elitismo y especifique cuál utiliza: Mejor individuo (p. ej., 1 élite). Porcentaje de mejores (p. ej., 5–10% de la población). Total (todos los individuos élite pasan sin alteración)
2) Operadores de selección (obligatorios) Implemente y ejecute el AG con cada uno de los siguientes: Ruleta. SUS (Stochastic Universal Sampling). Torneo: (indique el tipo de torneo utilizado)
3) Línea base (sugerida) Incluya Selección Aleatoria (uniforme) como testigo para ilustrar presión de selección baja y sus implicaciones en convergencia.

Control de variables Ejecute para cada entorno de prueba los tres operadores de selección con y sin elitismo. Mantenga constantes (para cada entorno de prueba) la población, número de generaciones, cruza, mutación y otros parámetros que correspondan, para aislar el efecto de la selección y del elitismo.

Criterio	Peso	Esperado	Implementado
1. Implementación de operadores de selección (Ruleta, SUS, Torneo)	20	Implementa los 3 operadores correctamente; documenta precisión de cada método; para Torneo especifica tipo, k, con/sin reemplazo y resolución de empates; Ruleta maneja aptitudes 0/negativas (normalización/shift) y SUS usa puntos equidistantes.	Torneo seleccionable, con y sin remplaza y resolución de empates.
2. Elitismo (tipo y parámetros)	20	Integra elitismo y declara explícitamente el tipo (mejor individuo / % mejores / total), el porcentaje/número, si saltan cruza/mutación, y el mecanismo de inserción en la siguiente generación; justifica su elección.	Elitismo con n% mejores como input.
3. Diseño experimental (control de variables + baseline)	15	Mantiene constantes población, generaciones, representación, fitness, cruza, mutación y tasas entre operadores; incluye baseline de Selección Aleatoria; detalla configuración.	

Criterio	Peso	Esperado	Implementado
5. Visualizaciones (consistencia y legibilidad)	15	Gráficas comparables, líneas de mejor y media, leyendas claras, títulos que incluyen operador y tipo de elitismo; figuras legibles.	
6. Resultados y discusión	10	Identifica la mejor combinación (operador + elitismo) y razona causas (presión de selección, diversidad, estancamiento); discute trade-offs (velocidad vs. calidad), efectos de elitismo y del torneo; reconoce limitaciones.	
7. Reproducibilidad (código)	10	Código limpio y modular; requisitos, comandos de ejecución, parámetros, es posible replicar todas las condiciones; estructura de carpetas clara.	
8. Informe (PDF) claridad y estructura	10	Breve y claro: problema, AG (representación/fitness/cruza/mutación), operadores y tipo de torneo, elitismo (tipo y parámetros), setup experimental, resultados, gráficas y tabla resumen, conclusiones; redacción formal.	

[1]:
```
from metazoo.bio.evolutionary import GeneticAlgorithm
from metazoo.bio.evolutionary.operators import selection, mutation, crossover
from metazoo.gym.mono import Function

# Set offline plotly
import plotly.io as pio
pio.renderers.default = "svg"

import pandas as pd
```

[2]:
```
targets = ['Sphere', 'Bukin', 'Himmelblau', 'EggHolder', 'Easom']
```

[3]:
```
encoding = 'binary'

if encoding == 'binary':
    mutation_function = mutation.flip_bit
else:
    mutation_function = mutation.gaussian
```

[31]:
```
results = []

for function in targets:
    fitness_function = Function(function, reverse=True)  # Usar el nombre␣
 ↪correcto
    for elitism in [None, 0.1, 0.3, 1.0]:
        for selection_ in [selection.uniform, selection.roulette, selection.
 ↪stochastic_universal_sampling, selection.rank, selection.tournament]:
```

```
            ga = GeneticAlgorithm(
                fitness_function=fitness_function,
                crossover_function=crossover.onepoint,
                mutation_function=mutation_function,
                selection_function=selection_,
                population_size=100,
                mutation_rate=0.1,
                crossover_rate=0.7,
                encoding=encoding,
                bounds=fitness_function.bounds,
                precision=3
            )
            ga.run(generations=100)
            result = {
                "func": function,
                "elitism": elitism,
                "selection": selection_.__name__,
                "best_fitness": ga.best_fitness,
                "best_individual": ga.best_individual.tolist() if hasattr(ga.
↪best_individual, "tolist") else ga.best_individual
            }
            results.append(result)
```

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

```
Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()
```

```
Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()
```

```
Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()
```

```
Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()
```

```
Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()
```

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

Output()

[32]:
```
summary = df.groupby(['func', 'selection', 'elitism']).agg(
    mean_best_fitness=('best_fitness', 'mean'),
    #min_best_fitness=('best_fitness', 'min'),
    #max_best_fitness=('best_fitness', 'max')
).reset_index()
summary.sort_values(by='mean_best_fitness', ascending=False, inplace=True)

for func in targets:
    func_summary = summary[summary['func'] == func]
    print(f"\nFunction: {func}")
    print(func_summary)
```

Function: Sphere

	func	selection	elitism	mean_best_fitness
72	Sphere	uniform	0.1	-36.171501
74	Sphere	uniform	1.0	-39.183866
73	Sphere	uniform	0.3	-41.407493
68	Sphere	stochastic_universal_sampling	1.0	-52.122081
63	Sphere	roulette	0.1	-52.147639
66	Sphere	stochastic_universal_sampling	0.1	-52.262616
65	Sphere	roulette	1.0	-52.358429
64	Sphere	roulette	0.3	-52.390408
67	Sphere	stochastic_universal_sampling	0.3	-52.409601
61	Sphere	rank	0.3	-52.428800
60	Sphere	rank	0.1	-52.428800
70	Sphere	tournament	0.3	-52.428800
71	Sphere	tournament	1.0	-52.428800
62	Sphere	rank	1.0	-52.428800
69	Sphere	tournament	0.1	-52.428800

Function: Bukin

	func	selection	elitism	mean_best_fitness
12	Bukin	uniform	0.1	-173.908169
13	Bukin	uniform	0.3	-209.147647
14	Bukin	uniform	1.0	-218.008083
4	Bukin	roulette	0.3	-226.663418
6	Bukin	stochastic_universal_sampling	0.1	-227.929052
5	Bukin	roulette	1.0	-228.634692
7	Bukin	stochastic_universal_sampling	0.3	-228.814755
8	Bukin	stochastic_universal_sampling	1.0	-229.030807
3	Bukin	roulette	0.1	-229.066768
0	Bukin	rank	0.1	-229.178785
2	Bukin	rank	1.0	-229.178785
1	Bukin	rank	0.3	-229.178785
11	Bukin	tournament	1.0	-229.178785
10	Bukin	tournament	0.3	-229.178785
9	Bukin	tournament	0.1	-229.178785

Function: Himmelblau

	func	selection	elitism	mean_best_fitness
57	Himmelblau	uniform	0.1	-291.300678
58	Himmelblau	uniform	0.3	-406.414281
59	Himmelblau	uniform	1.0	-462.138639
49	Himmelblau	roulette	0.3	-581.678727
46	Himmelblau	rank	0.3	-590.570478
45	Himmelblau	rank	0.1	-610.000000
55	Himmelblau	tournament	0.3	-610.000000
48	Himmelblau	roulette	0.1	-876.279589
52	Himmelblau	stochastic_universal_sampling	0.3	-880.683419

```
51  Himmelblau  stochastic_universal_sampling      0.1        -881.957578
50  Himmelblau                      roulette      1.0        -884.516545
53  Himmelblau  stochastic_universal_sampling      1.0        -887.227049
56  Himmelblau                    tournament      1.0        -890.000000
54  Himmelblau                    tournament      0.1        -890.000000
47  Himmelblau                          rank      1.0        -890.000000
```

Function: EggHolder

```
        func                      selection  elitism  mean_best_fitness
34  EggHolder                      roulette      0.3        -605.636285
42  EggHolder                       uniform      0.1        -607.260819
33  EggHolder                      roulette      0.1        -641.532988
44  EggHolder                       uniform      1.0        -677.831120
36  EggHolder  stochastic_universal_sampling      0.1        -684.392821
43  EggHolder                       uniform      0.3        -702.117539
37  EggHolder  stochastic_universal_sampling      0.3        -716.081288
35  EggHolder                      roulette      1.0        -717.932192
31  EggHolder                          rank      0.3        -786.523170
41  EggHolder                    tournament      1.0        -894.237710
39  EggHolder                    tournament      0.1        -894.551529
32  EggHolder                          rank      1.0        -894.578516
30  EggHolder                          rank      0.1        -894.578900
38  EggHolder  stochastic_universal_sampling      1.0       -1045.139947
40  EggHolder                    tournament      0.3       -1049.131624
```

Function: Easom

```
     func                      selection  elitism  mean_best_fitness
28  Easom                       uniform      0.3       0.000000e+00
15  Easom                          rank      0.1     -5.706807e-211
27  Easom                       uniform      0.1      -1.382892e-67
17  Easom                          rank      1.0      -2.120799e-29
29  Easom                       uniform      1.0      -4.553275e-27
16  Easom                          rank      0.3      -7.871790e-14
22  Easom  stochastic_universal_sampling      0.3      -5.061701e-05
18  Easom                      roulette      0.1      -5.084205e-05
23  Easom  stochastic_universal_sampling      1.0      -5.159888e-05
20  Easom                      roulette      1.0      -5.159913e-05
19  Easom                      roulette      0.3      -5.159932e-05
21  Easom  stochastic_universal_sampling      0.1      -7.579514e-03
25  Easom                    tournament      0.3      -9.001792e-03
26  Easom                    tournament      1.0      -9.005667e-03
24  Easom                    tournament      0.1      -9.005667e-03
```